CUSTOMER CARE REGISTRY

Centralized Customer Support & Complaint Management Platform

Project Documentation

Category: Nonprofit CRM | Domain: Full-Stack Web Development (MERN Stack)

Team Lead: Sameer Shaik

1. Introduction

1.1 Project Title

Customer Care Registry – A Centralized Platform for Managing Customer Interactions, Complaints, and Feedback

1.2 Team Members

Name	Role
Sameer Shaik	Team Lead
Jordan Tappita	Member
Padamata Navadeep	Member
Tharuni Veerla	Member
Yugandhar Thatapudi	Member

2. Project Overview

2.1 Purpose

A Customer Care Registry is a centralized system that records and manages customer interactions, issues, and feedback. It enables organizations to streamline support processes, track inquiries, and analyze trends to enhance service quality. By maintaining a comprehensive history of customer interactions, the registry ensures consistency in responses, identifies recurring pain points, and aids in proactive issue resolution.

With data-driven insights, support teams can refine training programs, optimize service protocols, and offer personalized support, ultimately improving customer satisfaction and loyalty. For a nonprofit organization in particular, the registry serves as a crucial tool for fostering trust, efficiency, and long-term relationships with the beneficiaries, donors, and volunteers it serves.

2.2 Features

- Centralized customer registry with full CRUD (create, view, edit, delete) and profile histories.
- Complaint management workflow with category, priority (Low/Medium/High), and status tracking (Pending → In Progress → Resolved → Closed).
- Automatic status timeline on every complaint, capturing who changed what and when.
- Interaction history log for calls, emails, meetings, and internal notes tied to each customer.
- Feedback module with 1–5 star ratings, comments, and aggregated satisfaction (CSAT) reporting.
- Role-based authentication (Admin and Support Agent) secured with JWT and bcrypt password hashing.
- Dashboard with live statistics, monthly complaint trends, category breakdowns, and a recent activity feed.
- Search, filter, sort, and pagination across customers and complaints (by name, complaint ID, status, priority, and assigned agent).

- Reports & analytics with CSV and PDF export for offline review and stakeholder reporting.
- Responsive, modern UI with toast notifications for every create/update/delete action.

2.3 Use-Case Scenarios

Scenario 1 – Rapid Complaint Intake and Assignment:

A support agent receives a call from a beneficiary about a delayed relief package. The agent quickly registers the customer (if new) and logs a complaint with High priority under the "Service Delay" category. The complaint is instantly assigned to the appropriate agent and appears on the dashboard's open-case count.

Scenario 2 – Escalation and Timeline Tracking:

A complaint remains unresolved for several days. A supervisor reviews the complaint's timeline, sees prior notes, updates the status to "In Progress" with a note describing the next action, and reassigns it to a senior agent — all changes are recorded automatically in the complaint history.

Scenario 3 – Monthly Reporting for Leadership:

At month end, a program manager opens the Reports page to review complaint volume by category, resolution rate, and average customer satisfaction score, then exports the data as a PDF/CSV to include in a board presentation.

3. Architecture

3.1 Frontend

The frontend is a single-page application built with React 18 and React Router 6. It uses a Context API-based AuthContext for global authentication state, an Axios service layer for all API communication, and Recharts for dashboard/report visualizations. The UI follows a sidebar + top-navbar layout with reusable components (cards, tables, modals, pagination, star ratings) and a custom, professional CSS theme — no external UI framework dependency was used.

3.2 Backend

The backend is a REST API built with Node.js and Express.js. Controllers are kept thin and delegate business logic to Mongoose models. Middleware handles JWT verification, role-based authorization, centralized error handling, and request validation (express-validator). Security hardening is provided via helmet and configured CORS.

3.3 Database Design (MongoDB Collections)

Collection	Purpose
Users	Stores agent/admin accounts with hashed passwords and role.
Customers	Stores customer profile details (name, phone, email, address, tags, status).
Complaints	Stores complaint details, category, priority, status, assigned agent, and status timeline.
InteractionHistory	Stores logged calls, emails, meetings, and notes per customer.
Feedback	Stores star ratings and comments, optionally linked to a specific complaint.

3.4 Data Storage

All application data is persisted in MongoDB using Mongoose schemas with validation rules (required fields, enums, and text indexes for search). The database can be run locally or hosted on MongoDB Atlas, requiring no additional flat-file storage.

3.5 Deployment

The frontend (React build) and backend (Node/Express API) are designed to be deployed independently — for example, the client on a static host (Vercel/Netlify) and the server on a Node-friendly host (Render/Railway), with MongoDB Atlas as the managed database layer. Environment variables keep configuration (database URI, JWT secret, client URL) separate from code.

4. Setup Instructions

4.1 Hardware Requirements

Component	Requirement
Processor	Intel i3 or above
RAM	4 GB (8 GB recommended)
Storage	2 GB free space
System Type	64-bit OS

4.2 Software Requirements

- Operating System: Windows 10/11, Linux, or macOS
- Node.js 18 or above
- MongoDB (local installation or a free MongoDB Atlas cloud cluster)
- Code Editor: VS Code (recommended)
- Web Browser: Google Chrome or Microsoft Edge

4.3 Required Packages

Backend: express, mongoose, bcryptjs, jsonwebtoken, express-validator, cors, helmet, morgan, dotenv.

Frontend: react, react-router-dom, axios, recharts, react-toastify, react-icons, jspdf, papaparse.

4.4 Installation Steps

Step 1 — Clone the repository:

```
git clone <GITHUB_REPO_LINK>
cd customer-care-registry
```

Step 2 — Backend setup (in one terminal):

```
cd server
npm install
cp .env.example .env
# then edit .env with your MongoDB URI and JWT secret
npm run seed
npm run dev
```

Step 3 — Frontend setup (in a second terminal):

```
cd client
npm install
cp .env.example .env
npm start
```

The backend runs at http://localhost:5000 and the frontend at http://localhost:3000.

5. Folder Structure

The project follows a clean, modular MERN structure split into client and server:

```
customer-care-registry/
|-- server/
|   |-- config/          # MongoDB connection
|   |-- controllers/     # Route handlers & business logic
|   |-- middleware/      # Auth, roles, error handling
|   |-- models/          # Mongoose schemas
|   |-- routes/          # Express routers
|   |-- utils/           # JWT helper, etc.
|   |-- seed/            # Sample data seed script
|   `-- server.js
`-- client/
    |-- public/
    `-- src/
        |-- components/  # Navbar, Sidebar, Table, Modal...
        |-- pages/       # Dashboard, Customers, Complaints...
        |-- context/     # Global auth state (AuthContext)
        |-- services/    # Axios API wrappers per resource
        |-- utils/       # Formatters, constants, exports
        `-- App.js
```

5.1 Key Components

- server.js: Configures Express, connects to MongoDB, mounts all resource routes, and applies global error handling.
- controllers/: Contain the logic for authentication, customers, complaints, interactions, feedback, and dashboard statistics.
- models/: Define schema validation, relationships (via ObjectId references), and auto-generated complaint IDs.
- client/src/context/AuthContext.js: Manages login, registration, logout, and session rehydration on refresh.
- client/src/services/: One file per resource (customerService, complaintService, feedbackService, etc.) wrapping Axios calls.

6. Running the Application

Once dependencies are installed and environment variables are set, start both servers:

```
cd server && npm run dev
cd client && npm start
```

The API runs at http://localhost:5000 and the client at http://localhost:3000. Open http://localhost:3000 in a browser, log in with a seeded account (or register a new one), and use the sidebar to navigate between the Dashboard, Customers, Complaints, Feedback, and Reports pages.

7. API / Route Documentation

Base URL: /api. All routes below (except auth) require an Authorization: Bearer <token> header.

7.1 Authentication Routes

Route	Method	Description
/auth/register	POST	Register a new user (support agent or admin).
/auth/login	POST	Authenticate and receive a JWT.
/auth/me	GET	Get the logged-in user's profile.

7.2 Customer & Complaint Routes

Route	Method	Description
/customers	GET / POST	List (search/filter/paginate) or create a customer.
/customers/:id	GET / PUT / DELETE	View, update, or delete (admin) a customer profile.
/complaints	GET / POST	List (search/filter/sort/paginate) or register a complaint.
/complaints/:id	GET / PUT / DELETE	View full timeline, update status/priority/agent, or delete (admin).
/interactions/customer/:id	GET	Get interaction history for a customer.
/feedback	GET / POST	List or submit customer feedback.
/feedback/reports	GET	Aggregated satisfaction (CSAT) report.
/dashboard/stats	GET	Card-level statistics for the dashboard.

Route	Method	Description
/dashboard/trends	GET	Monthly trend and category/priority breakdowns for charts.

7.3 Example Request (POST /complaints)

```
{
  "customer": "665f1a2b3c4d5e6f7a8b9c0d",
  "subject": "Delayed relief package delivery",
  "description": "Customer reports a two-week delay in receiving supplies.",
  "category": "Service Delay",
  "priority": "High",
  "assignedAgent": "665f1a2b3c4d5e6f7a8b9c1e"
}
```

7.4 Example Response

```
{
  "success": true,
  "data": {
    "complaintId": "CMP-000026",
    "status": "Pending",
    "priority": "High",
    "timeline": [
      { "status": "Pending", "note": "Complaint registered" }
    ]
  }
}
```

8. Authentication & Security

Unlike a purely internal decision-support tool, the Customer Care Registry implements full authentication and role-based access control, since it manages sensitive customer and complaint data:

- Passwords are hashed with bcrypt (10 salt rounds) before being stored — plaintext passwords are never persisted.
- JSON Web Tokens (JWT) are issued on login/registration and required on every protected route via the Authorization header.
- Two roles are supported: Admin (full access, including delete operations) and Support Agent (day-to-day create/update access).
- Destructive actions — deleting customers, complaints, feedback, or interactions — are restricted to Admin users only.
- All write endpoints validate input using express-validator before touching the database.
- helmet and a configured CORS policy protect the API from common HTTP vulnerabilities and restrict cross-origin access to the trusted client URL.
- Sensitive configuration (database URI, JWT secret) is stored in environment variables, never committed to source control.

9. User Interface

The interface is organized around a persistent sidebar (Dashboard, Customers, Complaints, Feedback, Reports) and a top navbar showing the logged-in user. Key screens include:

- Dashboard: stat cards (total customers, total complaints, open/closed cases), a monthly complaint trend line chart, a category breakdown pie chart, and a recent activity feed.
- Customers: a searchable, filterable, paginated table with add/edit modals and a detailed customer profile page (complaints, interaction timeline, and feedback tabs).
- Complaints: a searchable, filterable table (by status, priority, agent) with a detail page showing the full status timeline.
- Feedback: star-rating submissions, comments, and an aggregated satisfaction summary.
- Reports: category and priority charts plus one-click CSV and PDF export of complaint data.

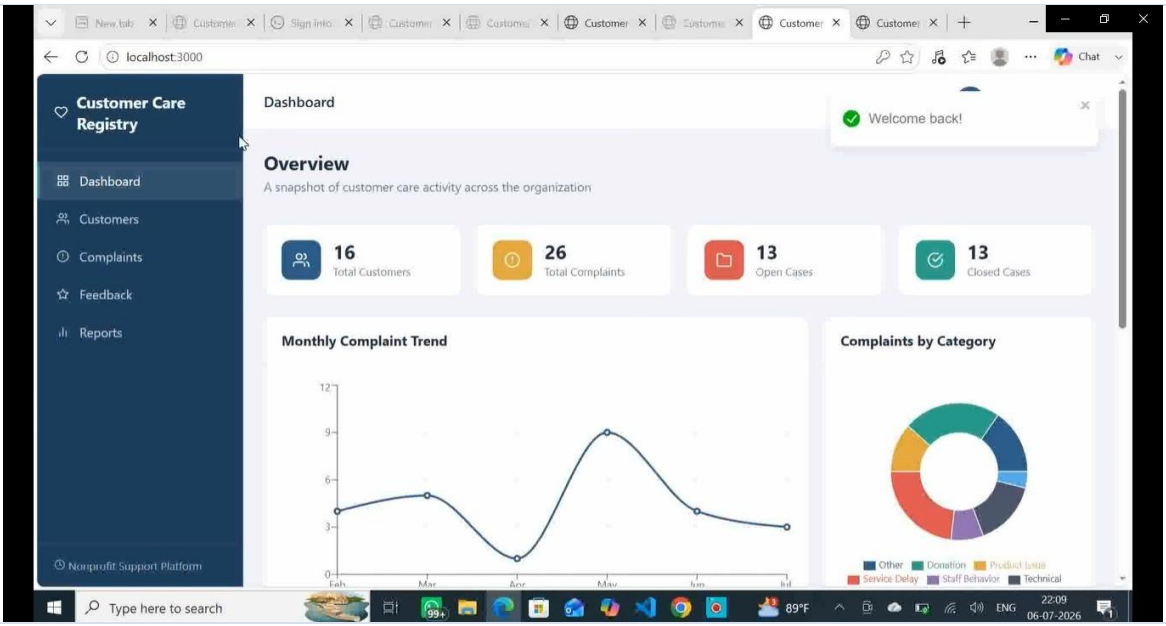

Fig 9.1 — Dashboard: live stats, monthly complaint trend, and category breakdown.

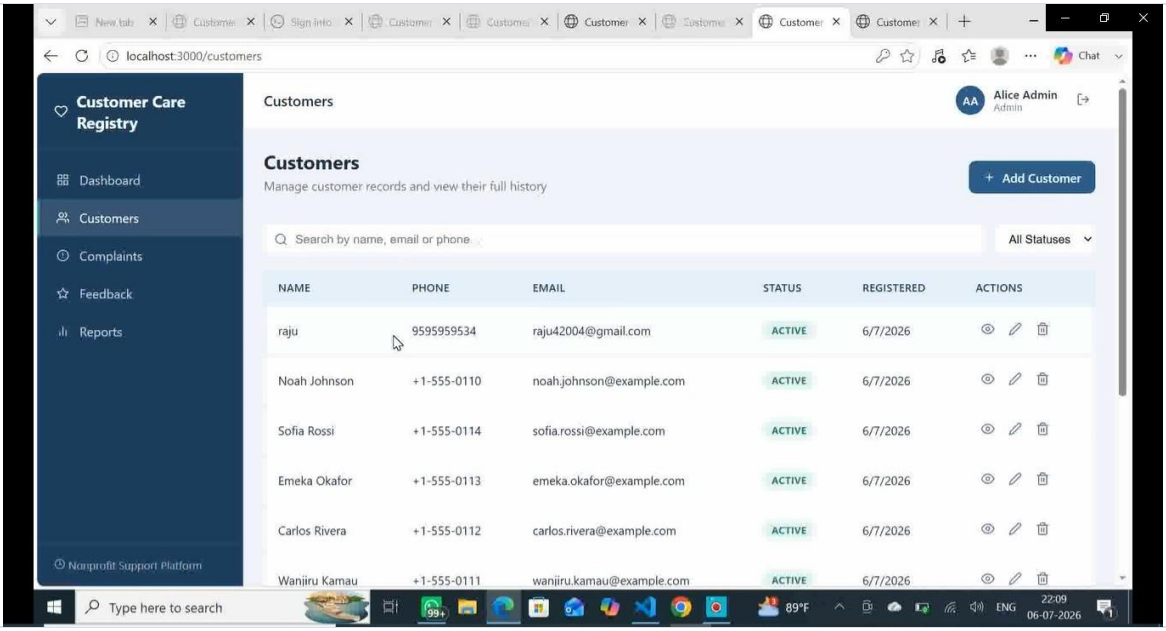

Fig 9.2 — Customers: searchable, filterable list with quick actions.

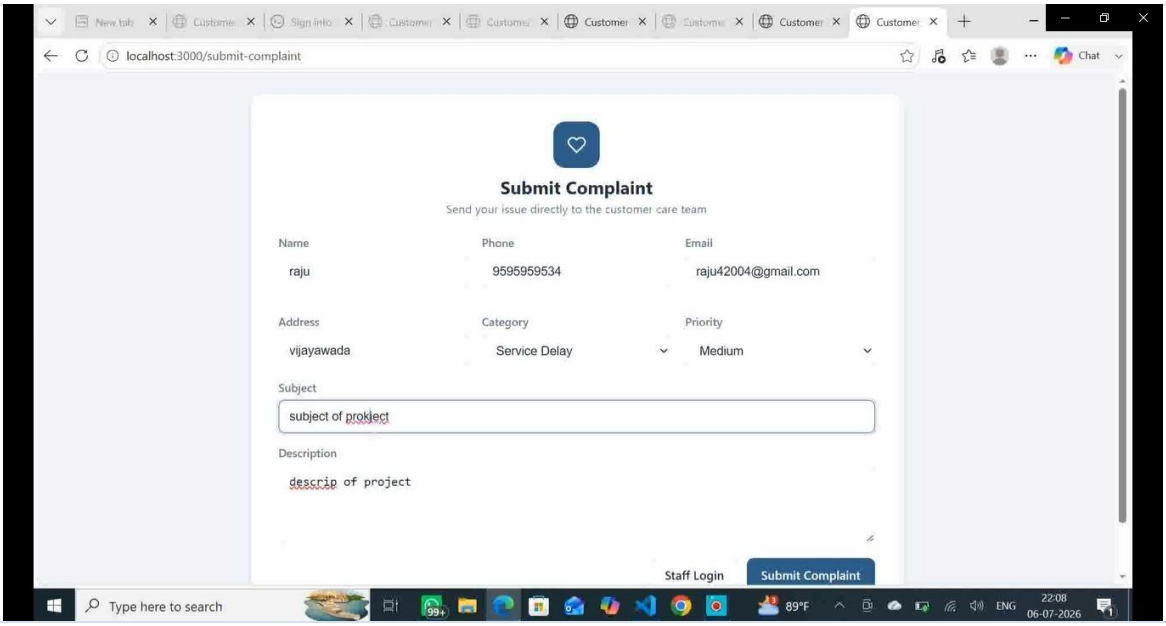

Fig 9.3 — Public complaint submission form for customers.

10. Testing

10.1 Functional Testing

- Manual verification of registration/login flows, including invalid credentials and expired tokens.
- CRUD testing for customers and complaints, confirming validation errors surface correctly in the UI.
- Verification that status changes on a complaint correctly append a new timeline entry with the acting user.
- Confirmation that role-based restrictions correctly block non-admin users from delete operations (403 response).

10.2 API Testing

- Health check endpoint (/api/health) verified to confirm the server and middleware chain load correctly.
- Search, filter, sort, and pagination parameters tested against the customers and complaints list endpoints.
- Dashboard aggregation endpoints (stats, trends, recent-activity) verified against seeded sample data.

10.3 Build Verification

- Backend: all server-side modules verified to load without errors; server starts and responds on the configured port.
- Frontend: production build (npm run build) completes successfully with no compile errors.

11. Screenshots or Demo

GitHub Repository: https://github.com/sameershaik16/customer-registry

Project Drive Folder (Docs & Media): https://drive.google.com/drive/folders/1i9J4JrzQREiYPFkc0igF-npSyqcmtuC4

The GitHub repository contains the full source code for both the client and server. The Drive folder holds supporting documentation, screenshots, and demo media for this project.

12. Known Issues

- No email notification system yet — status changes are visible in-app only, not sent via email.
- Bulk import of customers (e.g., via CSV upload) is not yet implemented; customers are added one at a time.
- Dashboard trend charts currently cover a fixed six-month window and are not yet user-configurable.
- No automated test suite (unit/integration tests) has been added yet; testing is currently manual.

13. Future Enhancements

- Add email/SMS notifications to customers and agents on complaint status changes.
- Support bulk customer and complaint import/export via CSV upload.
- Add configurable dashboard date ranges and downloadable dashboard snapshots.
- Introduce automated testing (Jest for backend, React Testing Library for frontend) and CI/CD pipelines.
- Add real-time updates (e.g., via WebSockets) so agents see new complaints and status changes instantly.
- Extend role-based access with a granular permissions system beyond Admin/Agent.